

Diseño de Software de Dispositivos Móviles

Unidad 8

Publicación y Distribución de Aplicaciones Móviles

App Store, Google Play, ASO, lanzamiento, marketing, métricas y mantenimiento post-lanzamiento

Asignatura: Diseño de Software de Dispositivos Móviles

Carrera: Tecnicatura Universitaria en Diseño de Software - FTyCA, UNCA

Docente: Mg. Ing. Carlos Acosta Parra

Actividad asociada: Simulación de publicación + plan de mejora post-lanzamiento

Entregable: Ficha técnica de publicación (descripción, capturas, keywords) + breve plan de mejoras futuras

Propósito de la unidad. Cerrar el recorrido de la asignatura mostrando cómo una aplicación móvil pasa de ser un proyecto funcional a un producto publicable, comunicable, monitoreable y sostenible en el tiempo.

Objetivos de aprendizaje

Al finalizar esta unidad, el estudiante será capaz de:

1. Explicar el proceso general de publicación de una aplicación móvil en App Store y Google Play.
2. Reconocer los requisitos técnicos, visuales, legales y de privacidad que deben prepararse antes de enviar una app a revisión.
3. Diseñar una ficha de publicación clara, persuasiva y coherente con el producto construido durante la asignatura.
4. Aplicar criterios básicos de ASO para mejorar la visibilidad y la conversión de una aplicación en tiendas digitales.
5. Planificar estrategias de lanzamiento, comunicación y promoción adecuadas a un proyecto académico o MVP.
6. Definir métricas de uso, calidad y retención para monitorear una app después de su publicación.
7. Organizar un plan de mantenimiento y mejoras post-lanzamiento basado en evidencia.
8. Preparar una simulación de publicación con capturas, keywords, descripción, checklist y roadmap breve.

Introducción

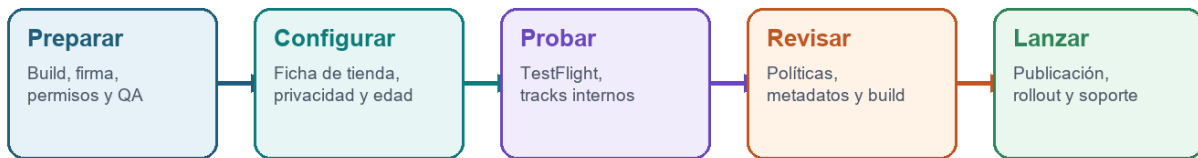
Las unidades anteriores recorrieron el camino completo de construcción de una aplicación móvil: comprensión del ecosistema, elección de alternativas tecnológicas, fundamentos de UI/UX, ideación, prototipado, desarrollo funcional e integración con servicios. La unidad 8 aborda el momento en que ese trabajo se convierte en una propuesta publicable.

Publicar una app no significa simplemente generar un archivo y subirlo a una tienda. Implica preparar una versión estable, cumplir políticas de plataforma, comunicar valor en pocos segundos, declarar datos y permisos, diseñar capturas, planificar soporte, medir el comportamiento real y sostener un ciclo de mejora.

En un proyecto académico no siempre se publicará efectivamente en App Store o Google Play. Sin embargo, simular seriamente la publicación obliga a pensar como equipo de producto: qué promete la app, a qué usuario se dirige, qué evidencia muestra, qué riesgos de privacidad existen, cómo se responderán los errores y qué se hará después del primer lanzamiento.

Idea central. Una app móvil no termina cuando compila: empieza a existir como producto cuando puede ser descubierta, comprendida, instalada, usada, evaluada, monitoreada y mejorada.

Del proyecto móvil a la publicación



Criterio central

Publicar no es subir un archivo: es preparar un producto verificable, entendible y medible.

Figura 1. Etapas conceptuales para llevar una app móvil desde el proyecto funcional hasta una publicación controlada.

1. Publicar como etapa de producto

Durante el desarrollo, el equipo suele concentrarse en que la aplicación funcione. En la publicación, la pregunta cambia: ¿la app está lista para encontrarse con usuarios reales, políticas reales, dispositivos reales y expectativas reales?

La publicación combina decisiones técnicas y decisiones de comunicación. Una build estable puede fracasar si su ficha de tienda no explica el valor. Una descripción atractiva puede ser rechazada si la app solicita permisos innecesarios o si la política de privacidad es insuficiente. Un lanzamiento con muchas instalaciones puede volverse problemático si no hay monitoreo de crashes ni un canal de soporte.

Tabla 1. Publicar una app requiere coordinar múltiples perspectivas.

Mirada	Pregunta principal	Ejemplos de decisiones
Técnica	¿La app funciona de forma estable en producción?	Firma, versión, pruebas, rendimiento, backend, compatibilidad.
UX/UI	¿La persona entiende y puede usar la app?	Onboarding, navegación, estados vacíos, errores, accesibilidad.
Negocio/producto	¿La app comunica valor y tiene una promesa clara?	Nombre, categoría, beneficios, público, posicionamiento.
Legal/privacidad	¿La app declara y protege los datos correctamente?	Permisos, política de privacidad, data safety, consentimiento.
Operación	¿El equipo puede mantener la app después del lanzamiento?	Soporte, métricas, roadmap, incidentes, actualizaciones.

1.1 Del vertical slice al release candidate

En las unidades 6 y 7 se trabajó con el concepto de vertical slice: una porción funcional que atraviesa interfaz, navegación, estado, datos e integración. Para publicar, ese vertical slice debe transformarse en un release candidate: una versión candidata a lanzamiento.

- No contiene funcionalidades incompletas visibles para el usuario final.
- Cuenta con manejo de errores, estados de carga y estados vacíos.
- Usa datos de prueba o producción de manera controlada, sin secretos expuestos.
- Incluye versión, nombre, ícono, permisos y configuración coherentes.
- Fue probada en más de un dispositivo, resolución o emulador.
- Tiene documentación básica para instalación, soporte y mantenimiento.

Regla práctica. No se publica para descubrir si la app funciona. Se publica una versión que ya fue probada para descubrir cómo se comporta con usuarios reales.

2. Preparación previa a la publicación

Antes de enviar una app a revisión conviene preparar una lista de control. Las tiendas evalúan tanto el archivo de la aplicación como la información declarada en la ficha, los permisos, la privacidad, la clasificación por edad y la coherencia entre lo que la app promete y lo que realmente hace.

Tabla 2. Checklist previo de preparación para publicación.

Elemento	Qué revisar	Riesgo si se ignora
Build de producción	Compilación release, firma, versión y configuración de entorno.	La app no puede instalarse, falla al iniciar o usa servicios incorrectos.
Identidad	Nombre, ícono, bundle id/package name y categoría.	Confusión, rechazo o imposibilidad de cambiar identificadores clave.
Permisos	Solicitar solo los necesarios y explicar su uso.	Desconfianza, abandono o incumplimiento de políticas.
Privacidad	Datos recolectados, finalidad, terceros y política visible.	Rechazo de tienda o pérdida de confianza.
Compatibilidad	Versiones mínimas, pantallas, orientación y dispositivos soportados.	Mala experiencia en dispositivos no probados.
QA funcional	Camino feliz, errores, datos vacíos, sesión expirada y desconexión.	Primeras reseñas negativas y soporte reactivo.
Accesibilidad	Contraste, tamaños táctiles, etiquetas y legibilidad.	Usuarios excluidos y mala calidad percibida.
Soporte	Email, sitio, FAQ o canal de contacto.	Usuarios sin respuesta ante problemas reales.

2.1 Firma, identificadores y versiones

Las plataformas móviles necesitan identificar de forma única cada aplicación. En Android se usa un package name; en iOS se usa un bundle identifier. Estos identificadores deben elegirse con cuidado porque acompañan a la app durante su vida útil.

También se debe distinguir entre versión visible para el usuario y número interno de build. La versión visible comunica cambios: por ejemplo 1.0.0, 1.1.0 o 2.0.0. El número de build permite que la tienda y el equipo diferencien compilaciones sucesivas.

Tabla 3. Elementos técnicos comparables entre Google Play y App Store.

Concepto	Android / Google Play	iOS / App Store
Identificador	Package name, por ejemplo com.equipo.app.	Bundle identifier, por ejemplo com.equipo.app.
Archivo de distribución	Android App Bundle (.aab), recomendado para Play.	Archivo generado desde Xcode y subido a App Store Connect.
Firma	Clave de firma y Play App Signing.	Certificados y provisioning profiles gestionados en Apple Developer.
Pruebas previas	Internal testing, closed testing, open testing.	TestFlight para testers internos o externos.
Revisión	Revisión de app, políticas, contenido y ficha.	App Review con revisión de build, metadatos y políticas.

2.2 Pruebas antes de enviar

El objetivo de las pruebas previas es reducir incertidumbre. No se busca garantizar que no exista ningún error, sino detectar problemas previsibles antes de exponer la app a usuarios, revisores o evaluadores.

- Instalar la app desde un paquete release, no solo desde el entorno de desarrollo.
- Probar inicio, navegación principal, cierre y reapertura.
- Verificar flujo sin conexión, errores del backend y sesión expirada.
- Comprobar que los permisos se soliciten en el momento adecuado.
- Revisar textos largos, pantallas pequeñas, teclado abierto y modo oscuro si aplica.
- Confirmar que no existan claves privadas, URLs de prueba o datos personales reales en la build.

3. App Store y Google Play: flujo general

App Store Connect y Google Play Console son las plataformas desde las que se gestionan versiones, fichas, pruebas, políticas, métricas, reseñas y distribución. Aunque cada una tiene pantallas y reglas propias, el flujo conceptual es similar.

1. Crear el registro de la aplicación en la consola correspondiente.
2. Configurar nombre, idioma principal, categoría, datos de contacto y edad/contenido.
3. Preparar la ficha de tienda: descripción, capturas, ícono, video si corresponde, keywords y textos promocionales.
4. Declarar privacidad, datos recolectados, permisos, anuncios, contenido sensible y público objetivo.
5. Subir la build firmada y asociarla a una versión.
6. Probar con usuarios internos o externos mediante canales de prueba.
7. Enviar a revisión o preparar rollout controlado.
8. Monitorear métricas, crashes, reseñas y adopción después de liberar.

Importante. Las políticas y requisitos de Apple y Google cambian con frecuencia. Para una publicación real, siempre se debe verificar la documentación oficial vigente antes de enviar la app.

3.1 Revisión y rechazo

Un rechazo no necesariamente indica que la app sea mala. Puede deberse a metadatos incompletos, credenciales de prueba faltantes, permisos sin justificación, errores al iniciar, contenido no permitido o diferencias entre la descripción y la funcionalidad real.

Tabla 4. Motivos de rechazo habituales y prevención.

Causa frecuente	Ejemplo	Cómo prevenirla
App incompleta	Botones sin función o pantallas de prueba visibles.	Enviar solo funcionalidades terminadas o claramente fuera de la ficha.
Credenciales faltantes	La revisión no puede acceder a una zona protegida.	Proveer usuario de prueba, instrucciones y datos mínimos.
Permisos excesivos	Solicitar ubicación sin usarla en el flujo principal.	Pedir permisos cuando sean necesarios y explicar finalidad.
Privacidad insuficiente	Recolectar email o ubicación sin política clara.	Declarar datos, finalidad, terceros y enlace a política.
Metadatos engañosos	Capturas muestran funciones no disponibles.	Alinear capturas, descripción y build real.
Errores técnicos	Crash al abrir o al iniciar sesión.	Probar release candidate en dispositivos y redes distintas.

4. Ficha de tienda y ASO

La ficha de tienda es la página pública desde la que una persona decide si instala o no una aplicación. Debe explicar valor, generar confianza y facilitar descubrimiento. ASO, App Store Optimization, es el conjunto de prácticas orientadas a mejorar visibilidad y conversión dentro de las tiendas.

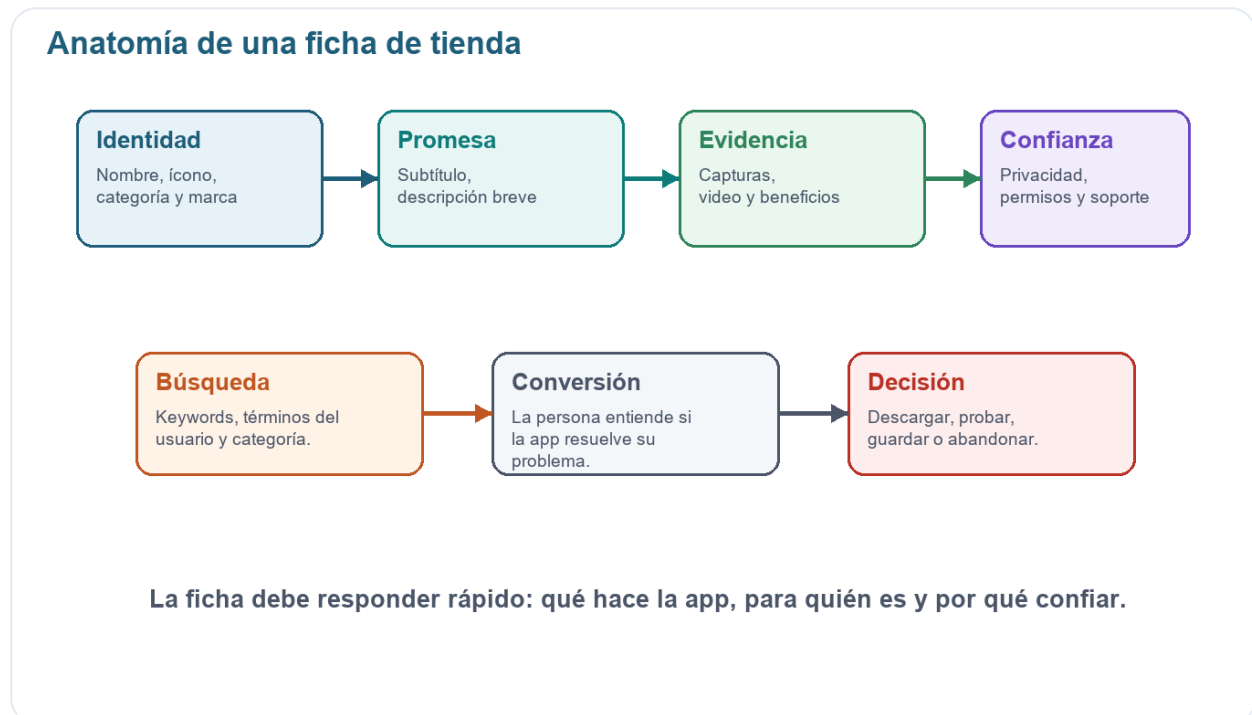


Figura 2. Componentes principales de una ficha de tienda y su relación con búsqueda, confianza y conversión.

4.1 Nombre, subtítulo y descripción

El nombre debe ser recordable, específico y coherente con el problema que resuelve. El subtítulo o descripción breve debe completar la promesa sin repetir lo mismo. La descripción larga debe explicar beneficios, funciones principales, público objetivo y condiciones relevantes.

Tabla 5. Elementos textuales de la ficha de publicación.

Elemento	Función	Buenas prácticas
Nombre	Identificar la app y comunicar categoría o beneficio.	Evitar nombres genéricos; usar una marca breve y pronunciable.
Subtítulo / breve	Resumir el valor en una línea.	Decir qué logra el usuario, no listar tecnología.
Descripción	Explicar problema, solución, funciones y confianza.	Primer párrafo fuerte; bullets legibles; tono humano.
Categoría	Ubicar la app en el contexto correcto.	Elegir según uso principal, no según deseo de visibilidad.
Keywords	Conectar intención de búsqueda con valor real.	Usar términos que el usuario diría, no solo jerga técnica.

4.2 Capturas y video

Las capturas no son decoración. Son evidencia visual de la experiencia. Deben mostrar pantallas reales, beneficios concretos y flujos relevantes. Una captura de login rara vez es la mejor primera imagen; suele ser más útil mostrar el resultado que la persona busca obtener.

- Mostrar la pantalla que mejor representa el valor principal de la app.
- Usar textos breves sobre la imagen si ayudan a interpretar el beneficio.
- Mantener coherencia visual entre capturas, ícono, marca y UI real.
- Incluir al menos un flujo completo: inicio, acción principal, resultado.
- Evitar capturas con datos sensibles, información de prueba confusa o errores visibles.
- Preparar tamaños y variantes según requisitos de cada tienda.

Criterio UX. Si la ficha necesita demasiado texto para explicar qué hace la app, probablemente la propuesta de valor o las capturas todavía no son claras.

4.3 Keywords y posicionamiento

Las keywords deben surgir del lenguaje del usuario, no solo del lenguaje del equipo. Una app de turnos médicos puede competir por términos como turnos, citas, clínica, salud, agenda, médico o recordatorios. La elección depende del país, público, categoría y problema principal.

ASO como ciclo de aprendizaje

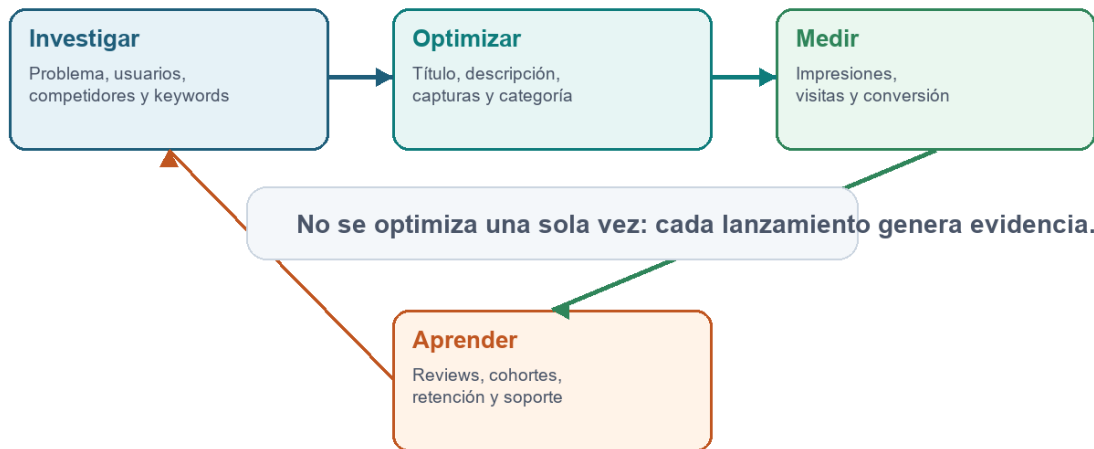


Figura 3. ASO entendido como ciclo continuo de investigación, optimización, medición y aprendizaje.

Tabla 6. Tipos de keywords útiles para una ficha de tienda.

Tipo de keyword	Ejemplo	Uso
Problema	organizar turnos, reservar cita, controlar gastos.	Conecta con una necesidad explícita.
Categoría	agenda médica, recetas, clima, finanzas personales.	Ubica la app dentro de una familia de soluciones.
Beneficio	recordatorios, ahorro, seguimiento, alertas.	Comunica resultado esperado.
Público	estudiantes, pacientes, docentes, emprendedores.	Ayuda a segmentar si el producto es específico.
Localización	Catamarca, Argentina, español.	Puede ser útil para apps territoriales o institucionales.

5. Privacidad, permisos y confianza

La publicación obliga a declarar cómo la app trata los datos. Esto incluye datos recolectados, finalidad, terceros, almacenamiento, permisos del dispositivo, cuentas, contenido generado por usuarios y mecanismos de eliminación cuando correspondan.

Desde el punto de vista del usuario, la confianza se construye con coherencia: la app pide lo que necesita, explica por qué, funciona sin abusar de permisos y no oculta información importante.

Tabla 7. Preguntas para preparar declaraciones de privacidad y permisos.

Área	Preguntas guía
Datos personales	¿Se recolectan nombre, email, ubicación, fotos, salud, pagos o identificadores?

Finalidad	¿Para qué se usa cada dato? ¿Es necesario para la funcionalidad principal?
Terceros	¿Se usan analytics, crash reporting, publicidad, mapas, pagos o backend externo?
Permisos	¿La app solicita cámara, ubicación, contactos, archivos o notificaciones? ¿Cuándo y por qué?
Retención	¿Cuánto tiempo se conservan datos? ¿Existe eliminación o cierre de cuenta si aplica?
Seguridad	¿Los datos viajan por HTTPS? ¿Los tokens se guardan en almacenamiento seguro?

Mínimo privilegio. Una app que pide menos permisos y explica mejor suele generar más confianza que una app que solicita todo al inicio por comodidad técnica.

6. Estrategias de lanzamiento y promoción

El lanzamiento es el primer contacto amplio entre la app y su público. Para un proyecto académico, puede consistir en una presentación, una demo pública, una landing page o una campaña interna. Para un producto real, puede incluir beta cerrada, prensa, redes sociales, email, comunidad y alianzas.

Tabla 8. Opciones de lanzamiento y promoción según contexto.

Estrategia	Cuándo sirve	Ejemplo
Beta cerrada	Antes de lanzar a todo el público.	10 a 30 usuarios prueban flujos críticos y reportan errores.
Landing page	Cuando se necesita explicar la propuesta fuera de la tienda.	Página con beneficios, capturas, FAQ y formulario de interés.
Contenido en redes	Cuando el público puede descubrir la app por comunidad.	Videos cortos mostrando casos de uso concretos.
Alianzas	Cuando la app resuelve un problema de una institución o grupo.	Convenio con escuela, clínica, comercio o área municipal.
Programa de referidos	Cuando el valor aumenta con más usuarios.	Invitar contactos, equipos o clientes.
Demo académica	Cuando el objetivo es evaluación y aprendizaje.	Video breve con flujo principal, publicación simulada y plan de métricas.

6.1 Mensaje de lanzamiento

Un buen mensaje de lanzamiento no enumera todo lo que la app tiene. Explica qué problema resuelve, para quién y qué acción se espera. Debe ser concreto y verificable.

Plantilla breve:

[Nombre de la app] ayuda a [usuario objetivo] a [resultado deseado] sin [dolor o fricción principal], mediante [función o enfoque principal].

Ejemplo:

TurnoClaro ayuda a pacientes de una clínica a reservar y recordar sus turnos sin depender de llamadas telefónicas, mediante agenda móvil, notificaciones y consulta de disponibilidad.

7. Métricas y monitoreo

Después de publicar, el equipo necesita evidencia. Las métricas no son números para decorar una presentación: son señales que ayudan a decidir qué mejorar, qué corregir, qué eliminar y qué priorizar.

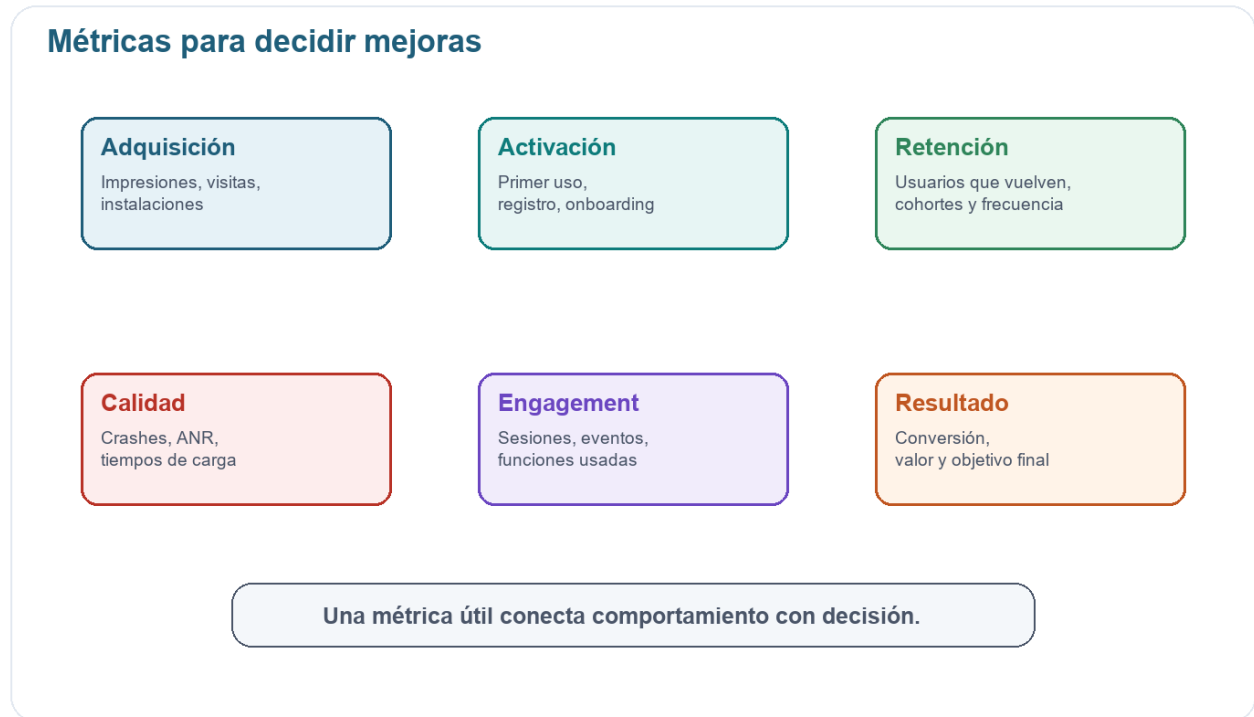


Figura 4. Grupos de métricas útiles para analizar una app publicada o en beta.

Tabla 9. Métricas frecuentes y decisiones asociadas.

Métrica	Qué indica	Decisión posible
Impresiones	Cuántas veces aparece la app en la tienda.	Ajustar keywords, categoría o promoción.
Conversión a instalación	Relación entre visitas a ficha e instalaciones.	Mejorar capturas, descripción, reseñas o propuesta de valor.
Activación	Usuarios que completan el primer flujo importante.	Revisar onboarding, permisos, registro o primera pantalla.
Retención	Usuarios que vuelven después de 1, 7 o 30 días.	Mejorar valor recurrente, recordatorios o contenido.
Crash-free users	Porcentaje de usuarios sin cierres inesperados.	Priorizar estabilidad antes de nuevas funciones.
Tiempo de carga	Velocidad percibida y técnica.	Optimizar API, cache, imágenes o estados de espera.
Reviews y rating	Percepción pública y problemas repetidos.	Responder soporte, corregir irritantes y ajustar expectativas.

7.1 Herramientas de monitoreo

Las propias tiendas ofrecen datos de adquisición, rendimiento y calidad. Además, pueden incorporarse herramientas de analytics, crash reporting y observabilidad. En un proyecto pequeño, no conviene medir todo: conviene medir lo que permite tomar decisiones.

Tabla 10. Fuentes posibles de monitoreo post-lanzamiento.

Herramienta / fuente	Qué aporta	Uso sugerido
App Store Connect	Analíticas, ventas, descargas, reviews, TestFlight.	Seguimiento de publicación iOS y calidad del lanzamiento.
Google Play Console	Instalaciones, Android vitals, reviews, pre-launch reports.	Monitoreo Android, crashes, ANR y conversión de ficha.
Firebase Analytics	Eventos, audiencias, funnels, retención.	Medir comportamiento dentro de la app.
Crashlytics / Sentry	Errores, crashes, trazas y versiones afectadas.	Priorizar fallas reales por impacto.
Encuestas / entrevistas	Motivos, expectativas y lenguaje del usuario.	Interpretar señales cuantitativas con contexto humano.

Cuidado. Medir actividad no equivale a medir valor. Muchas sesiones pueden indicar interés, pero también confusión si la persona repite pasos porque no logra completar la tarea.

8. Mantenimiento y ciclo de vida post-lanzamiento

Publicar abre una nueva etapa: soporte, corrección de errores, actualizaciones de plataforma, cambios de políticas, mejoras de usabilidad y evolución del producto. Una app abandonada pierde confianza aunque su primera versión haya sido correcta.



Figura 5. El ciclo post-lanzamiento combina versiones, soporte, monitoreo y priorización.

8.1 Tipos de actualización

Tabla 11. Tipos de actualización después del lanzamiento.

Tipo	Objetivo	Ejemplo
Corrección crítica	Resolver un error que impide usar la app.	Crash al iniciar sesión en Android 15.
Hotfix menor	Corregir problema puntual de baja superficie.	Texto incorrecto, validación, enlace roto.
Mejora de usabilidad	Reducir fricción o confusión observada.	Simplificar onboarding o mensajes de error.
Nueva funcionalidad	Agregar valor nuevo alineado al roadmap.	Favoritos, historial, filtros avanzados.
Actualización técnica	Mantener compatibilidad y seguridad.	Actualizar SDK, permisos, librerías o APIs.
Cambio de contenido	Actualizar textos, imágenes o configuración.	Nuevas capturas, categorías, FAQ o promociones.

8.2 Priorización de mejoras

Después del lanzamiento aparecen muchas ideas: deseos del equipo, comentarios de usuarios, errores, oportunidades de marketing y cambios técnicos. Para priorizar, conviene combinar impacto, urgencia, esfuerzo y evidencia.

Tabla 12. Señales comunes y prioridad de mejora.

Señal	Impacto	Prioridad probable
Crash frecuente en flujo principal	Impide uso y genera reseñas negativas.	Muy alta.
Muchos usuarios abandonan en registro	Bloquea activación.	Alta.
Pedido de función por pocos usuarios avanzados	Puede aportar valor, pero requiere validar alcance.	Media.
Cambio estético sin evidencia	Puede consumir tiempo sin mejorar resultado.	Baja.
Política de tienda próxima a cambiar	Puede impedir actualizar o mantener disponible la app.	Alta.

Plan de mejora basado en evidencia

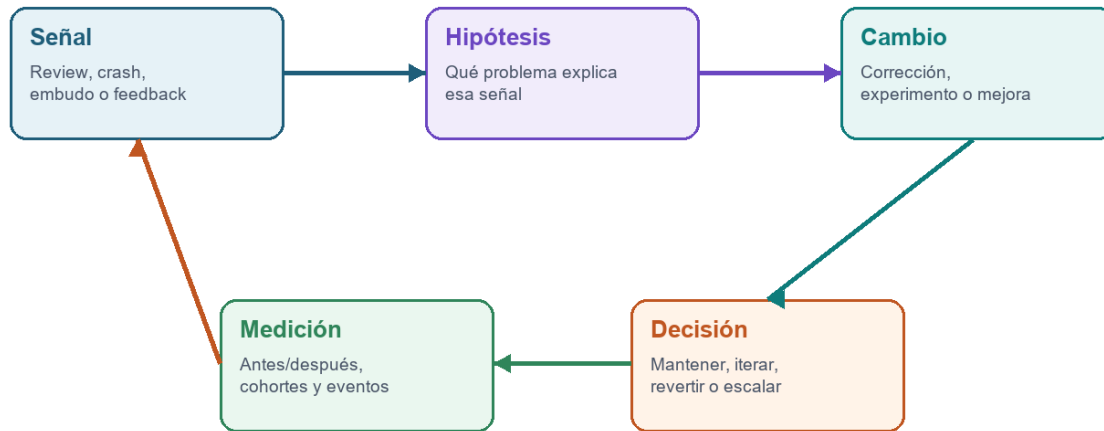


Figura 6. Plan de mejora basado en señal, hipótesis, cambio, medición y decisión.

Bibliografía y recursos de referencia

- Apple Developer. App Store Connect Help: preparación, envío, TestFlight, metadatos y revisión de apps.
- Apple Developer. App Review Guidelines: criterios oficiales de revisión para publicación en App Store.
- Google Play Console Help. Create and set up your app; prepare and roll out releases; store listing and policy declarations.
- Google Play Console Help. Android vitals and app quality: monitoreo de crashes, ANR, rendimiento y calidad.
- Google Play Academy. Recursos de aprendizaje sobre lanzamiento, ficha de tienda, políticas y crecimiento.
- Tidwell, J.; Brewer, C.; Valencia, A. Designing Interfaces. O'Reilly Media, 2019.
- Dakić, M. Mobile App Development for Businesses: Create a Product Roadmap and Digitize Your Operations. Apress, 2023.
- Yablonski, J. Laws of UX. O'Reilly Media, 2024.
- Documentación oficial del framework utilizado por el equipo: Flutter, React Native, Ionic, Android, iOS u otra alternativa definida en el proyecto.